Les modèles en Django

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte 3

II. Création des modèles 3

A. Création d'un premier modèle 3

B. Migration de la base de données 4

C. Django ORM et le shell Django 5

D. Exercice : Quiz 6

III. Rechercher à l'aide des QuerySets 7

A. Recherche simple 7

B. Recherche multicritères et objet Q 8

C. Exercice : Quiz 9

IV. Relations multiples entre les modèles 10

A. Utiliser des clés étrangères 10

B. Relations plusieurs-à-plusieurs 11

C. Gérer les modèles via l'interface d'administration 11

D. Exercice : Quiz 12

V. L'essentiel 13

VI. Auto-évaluation 13

A. Exercice 13

B. Test 13

Solutions des exercices 15

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60 minutes

Environnement de travail : Python 3.10+, Django 4.2+, un éditeur de texte (PyCharm, VS Code, etc.), un navigateur web.

Pré requis : bases en Python, notions de base du développement web (HTML, CSS), connaissances des principes de base Django.

Contexte

S'engager dans la sphère du développement web à l'aide de Django est un voyage exaltant, truffé de nouvelles connaissances et de réalisations significatives. Ce framework, estimé par une multitude de développeurs à travers le monde, offre un ensemble d'outils exhaustif pour la création de sites web de manière rapide et sécurisée. Django favorise le principe DRY (Don't Repeat Yourself), encourageant la réutilisation du code et l'élimination de la redondance, tout en apportant une structure de projet organisée qui permet de naviguer avec clarté dans le développement d'applications web complexes.

Une caractéristique qui distingue Django est son approche uniforme et bien organisée du développement web. Un projet Django suit une structure précise qui stimule la cohérence et la modularité. Comprendre cette structure est fondamental pour naviguer avec aisance à travers vos projets. Dans ce cours, nous allons lever le voile sur ces aspects, en décomposant chaque composant et son rôle. Il s'agit du point de départ pour maîtriser Django et évoluer vers un développeur Django accompli.

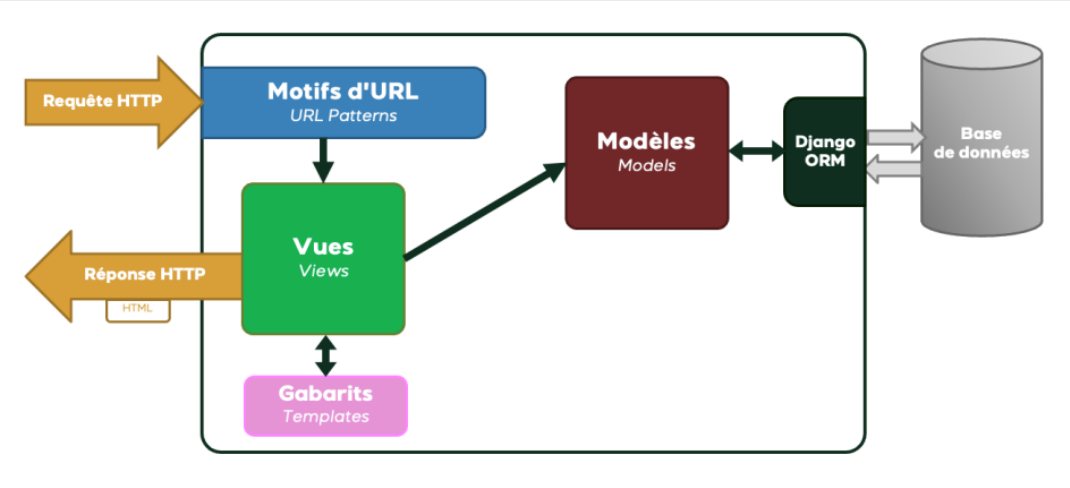
Plus spécifiquement, nous allons nous concentrer sur les modèles en Django, des éléments essentiels pour la gestion des données de votre application. La création, la modification et la migration des modèles représentent des compétences cruciales à acquérir pour travailler efficacement avec Django. Nous allons vous guider à travers ces processus, vous offrant une compréhension approfondie qui vous permettra d'interagir avec votre base de données de manière fluide et sécurisée.

II. Création des modèles

A. Création d'un premier modèle

Rappel **Les modèles dans l'architecture MTV**

Dans l'architecture MTV, les modèles représentent la structure et la logique des données de l'application. Ils correspondent aux tables de la base de données et définissent les champs (colonnes) et les comportements des données (lignes).



Django, l'un des principaux frameworks de développement web en Python, fait appel aux « *modèles* » pour représenter les données de votre application. Ces modèles, définis sous forme de classes Python, offrent une interface hautement intuitive pour interagir avec votre base de données. Les modèles de Django abstraient les détails de bas niveau de SQL et vous permettent de manipuler les données de votre application comme s'il s'agissait de simples objets Python.

Méthode

La création d'un modèle en Django est un processus précis qui nécessite une attention particulière. Dans Django, un modèle est la représentation unique, spécifique et détaillée d'une source de données ; il s'agit en quelque sorte du plan à partir duquel votre base de données sera construite.

Un modèle est défini en Python comme une classe qui hérite de *django.db.models.Model*. Chaque attribut de cette classe représente un champ spécifique dans la base de données. Ces champs indiquent le type de données à stocker (par exemple, texte, nombre, date, etc.) et peuvent inclure d'autres paramètres tels que l'exigence d'unicité.

En utilisant Django, vous pouvez créer un modèle assez rapidement. Prenons par exemple le cas où nous souhaitons créer un modèle pour représenter un étudiant. Nous souhaitons inclure des informations telles que le nom, l'e-mail et la date de naissance de l'étudiant. Dans ce cas, la définition du modèle pourrait ressembler à ce qui suit :

```
1 from django.db import models
2
3 class Etudiant(models.Model):
4     nom = models.CharField(max_length=100)
5     prenom = models.CharField(max_length=100)
6     email = models.EmailField(unique=True)
7     date_naissance = models.DateField()
```

Attention

Dans Django, la création de modèles nécessite une compréhension approfondie de la syntaxe et des conventions utilisées. Chaque attribut de classe dans votre modèle correspond à un champ spécifique de la table dans la base de données. Ces attributs sont définis en utilisant des instances de classes de champ que Django fournit pour divers types de données. Par exemple, pour représenter des données de type caractère, vous utiliserez CharField, tandis que pour représenter des dates, vous utiliserez DateField. Il est important de bien comprendre que ces types de champ sont spécifiques à Django et peuvent différer des types de données traditionnels utilisés dans les bases de données SQL.

Il convient également de noter que la convention en Django est de nommer les attributs de modèle en minuscules, en utilisant des underscores pour séparer les mots si nécessaire. Cela contribue à maintenir la lisibilité du code et facilite sa maintenance.

Enfin, bien que Django offre une grande flexibilité dans la définition de vos modèles, il est essentiel de réfléchir soigneusement à la structure de vos données avant de définir vos modèles. Un modèle bien conçu peut grandement faciliter le développement de votre application, tandis qu'un modèle mal conçu peut entraîner des complications et des difficultés en cours de route.

B. Migration de la base de données

Tous droits réservés à STUDI - Reproduction interdite

Django fournit une gamme de commandes de gestion des migrations pour aider les développeurs à gérer leurs bases de données. Voici les deux principales commandes que vous utiliserez :

- `makemigrations` : cette commande analyse vos définitions de modèles, détecte les changements par rapport à la version précédente et crée de nouvelles migrations pour refléter ces changements. C'est généralement la première commande que vous exécuterez après avoir créé ou modifié un modèle.
- `migrate` : cette commande applique les migrations non appliquées (c'est-à-dire les migrations créées, mais non encore exécutées) à votre base de données. En d'autres termes, elle exécute les instructions contenues dans vos fichiers de migration pour modifier la structure de la base de données.

Attention

Utilisez les commandes de migration avec précaution et discernement. En effet, ces outils peuvent apporter des modifications substantielles à la structure de votre base de données, ce qui peut avoir des conséquences importantes si les choses ne se passent pas comme prévu.

- `makemigrations` : cette commande doit être utilisée après avoir soigneusement vérifié et validé vos définitions de modèles. Gardez à l'esprit qu'elle crée des instructions de migration basées sur les différences entre vos modèles actuels et les précédents. Si vos modèles contiennent des erreurs ou des omissions, celles-ci se répercuteront sur les migrations générées.
- `migrate` : avant d'exécuter cette commande, assurez-vous d'avoir vérifié les migrations qui seront appliquées. Toute modification de la structure de la base de données doit être effectuée de manière réfléchie, car une fois la migration appliquée, il peut être complexe de revenir en arrière sans une bonne gestion des versions.

En bref, ne considérez jamais les commandes de migration comme des opérations anodines. Chaque migration est une modification potentielle de votre base de données, et chaque modification doit être faite avec prudence et préparation appropriée.

Rollback de migrations

Un rollback, en termes de gestion de base de données, est une opération qui annule une série de transactions ou de modifications. Dans le contexte des migrations de Django, un rollback consiste à annuler une ou plusieurs migrations qui ont été appliquées à votre base de données.

Dans Django, vous pouvez effectuer un rollback de migrations en utilisant la commande migrate suivie du nom de l'application et du numéro de la migration jusqu'où vous souhaitez revenir. Par exemple, pour annuler toutes les migrations d'une application appelée *myapp*, vous utiliseriez la commande :

```
./manage.py migrate myapp zero
```

Si vous voulez annuler la dernière migration appliquée, vous pouvez simplement exécuter la commande migrate suivie du nom de l'application et du nom de la migration précédente. Par exemple, si vous avez une migration nommée *0004_add_telephone* et que vous voulez revenir à la migration *0003_add_birthdate*, vous utiliseriez la commande :

```
migrate myapp 0003_add_birthdate
```

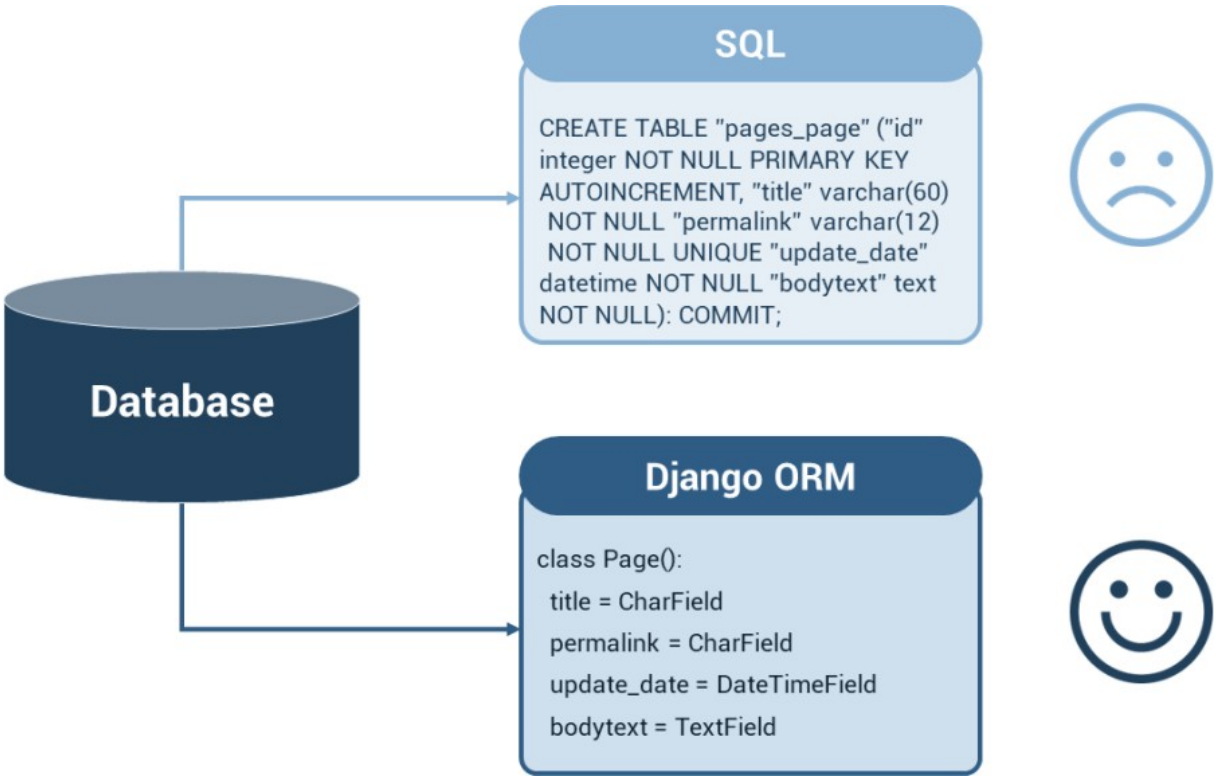
Il est important de noter que le rollback des migrations peut avoir des conséquences importantes, notamment la perte de données. Par conséquent, il convient de l'utiliser avec prudence et de toujours avoir une sauvegarde de la base de données avant de procéder à un rollback.

C. Django ORM et le shell Django

Tous droits réservés à STUDI - Reproduction interdite

L'un des avantages les plus importants de l'ORM (Object-Relational Mapping) de Django réside dans sa capacité à simplifier grandement les opérations sur la base de données. Avec l'ORM de Django, les opérations courantes comme l'ajout, la modification ou la suppression d'enregistrements se traduisent en opérations simples sur les objets Python. Il n'est plus nécessaire d'écrire du code SQL à la main, ce qui réduit le risque d'erreurs de syntaxe SQL et rend le code plus lisible et maintenable.

De plus, l'ORM de Django prend en charge un grand nombre de bases de données, comme PostgreSQL, MySQL, SQLite, et Oracle. Ainsi, vous pouvez changer de base de données simplement en modifiant une ligne dans vos paramètres de configuration, sans avoir à réécrire votre code. Cela rend Django particulièrement flexible et évolutif, tout en offrant une couche d'abstraction qui protège votre application contre les détails spécifiques de la base de données.



D. Exercice : Quiz

[solution n°1 p.17]

Question 1

Qu'est-ce qui est représenté par une classe dans un modèle Django ?

- ○ Une requête à la base de données
- ○ Une colonne dans une table de la base de données
- ○ Une table dans la base de données

Question 2

Dans la définition d'un modèle Django, que représente un attribut de classe ?

- ○ Un enregistrement dans la table de la base de données
- ○ Une colonne dans la table de la base de données
- ○ Une base de données entière

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Quelle est la première commande que vous exécuterez après avoir créé ou modifié vos modèles Django ?

- makemigrations
- migrate
- syncdb

Question 4

Comment pouvez-vous ajouter un champ de numéro de téléphone à un modèle d'Utilisateur existant ?

- En ajoutant une nouvelle instance de models.CharField au modèle Utilisateur
- En modifiant le champ email pour y inclure un numéro de téléphone
- En supprimant le champ email et en le remplaçant par un champ de numéro de téléphone

Question 5

Qu'est-ce qu'un rollback dans le contexte des migrations de Django ?

- Une façon de créer une nouvelle base de données
- Un moyen d'annuler les modifications apportées à la base de données par une migration
- Un moyen de sauvegarder votre base de données

Question 6

Quelle est la commande pour effectuer un rollback à une migration spécifique dans Django ?

- python manage.py migrate <nom_app> <nom_migration>
- python manage.py rollback <nom_app> <nom_migration>
- python manage.py revert <nom_app> <nom_migration>

III. Rechercher à l'aide des QuerySets

A. Recherche simple

Définition **Les QuerySets avec Django**

Un QuerySet, en Django, est un objet qui représente une collection d'objets de base de données. Il peut contenir zéro, un ou plusieurs filtres, et vous donne la possibilité de filtrer, de trier, et de manipuler des ensembles de données provenant de votre base de données de manière très performante.

L'efficacité des QuerySets

L'un des atouts majeurs des QuerySets réside dans leur « paresse » (laziness). En d'autres termes, un QuerySet ne va pas exécuter de requête sur la base de données tant que vous n'avez pas besoin du résultat. Ainsi, vous pouvez chaîner plusieurs opérations (filtres, tris, etc.) sur un QuerySet sans qu'aucune requête ne soit réellement envoyée à la base de données, jusqu'à ce que vous ayez besoin du résultat. Cela permet d'optimiser l'efficacité de vos interactions avec la base de données.

Tous droits réservés à STUDI - Reproduction interdite

Méthode **Les commandes usuelles des QuerySets**

Les QuerySets sont extrêmement flexibles et offrent de nombreuses commandes pour manipuler les données stockées dans votre base de données. Voici un aperçu des commandes les plus couramment utilisées :

`all()` : cette commande retourne un QuerySet contenant tous les éléments d'un modèle donné. Exemple :

```
all_objects = MyModel.objects.all()
```

`filter()` : cette commande retourne un QuerySet qui inclut uniquement les éléments qui correspondent aux critères spécifiés. Exemple :

```
filtered_objects = MyModel.objects.filter(field="value")
```

`exclude()` : cette commande retourne un QuerySet qui exclut les éléments qui correspondent aux critères spécifiés. Exemple :

```
excluded_objects = MyModel.objects.exclude(field="value")
```

`order_by()` : cette commande retourne un QuerySet qui est trié selon les champs spécifiés. Exemple :

```
sorted_objects = MyModel.objects.order_by('field')
```

`get()` : cette commande retourne l'objet qui correspond exactement aux critères spécifiés. Si aucun ou plusieurs objets correspondent, cela provoque une erreur. Exemple :

```
single_object = MyModel.objects.get(id=1)
```

`values()` : cette commande retourne un QuerySet qui renvoie des dictionnaires au lieu d'instances de modèle. Exemple :

```
value_objects = MyModel.objects.values('field1', 'field2')
```

`values_list()` : cette commande retourne un QuerySet qui renvoie des tuples lorsqu'itéré. Chaque tuple contient la valeur du champ correspondant à la position du champ dans la liste de paramètres. Exemple :

```
value_list_objects = MyModel.objects.values_list('field1', 'field2')
```

`annotate()` : cette commande retourne un QuerySet qui effectue des calculs d'agrégation sur les objets renvoyés. Exemple :

```
from django.db.models import Count
annotated_objects = MyModel.objects.values('field').annotate(count=Count('field'))
```

`aggregate()` : cette commande effectue un calcul sur l'ensemble du QuerySet renvoyé et renvoie un dictionnaire des valeurs agrégées. Exemple :

```
from django.db.models import Avg
aggregated_objects = MyModel.objects.aggregate(Avg('field'))
```

Attention **La performance des QuerySets**

Malgré leur simplicité d'utilisation, les QuerySets sont des outils puissants et efficaces pour interagir avec la base de données. Cependant, il est important de les utiliser de manière réfléchie pour éviter les problèmes de performance.

Par exemple, l'utilisation abusive de la méthode `all()` peut entraîner des requêtes volumineuses et potentiellement coûteuses en matière de temps de traitement. De même, l'enchâssement de requêtes (c'est-à-dire, l'exécution de requêtes à l'intérieur de boucles) peut aussi être une source de problèmes de performance.

B. Recherche multicritères et objet Q

Tous droits réservés à STUDI - Reproduction interdite

Tout comme en SQL, Django ORM fournit via les QuerySets la possibilité d'effectuer des recherches multicritères de type *AND*, *OR* ou bien encore l'opération de négation *NOT*.

Pour ces deux derniers types de recherche, Django met à disposition une classe nommée *Q* qui permet de concaténer les filtres de recherche.

C. Exercice : Quiz

[solution n°2 p.18]

Question 1

Quel est le rôle principal d'un QuerySet dans Django ?

- Représenter une requête SQL sur un modèle
- Générer des formulaires automatiquement
- Créer des vues basées sur des classes
- Gérer les fichiers statiques

Question 2

Quelle méthode est utilisée pour récupérer tous les objets d'un modèle sous forme de QuerySet ?

- get()
- all()
- filter()
- select()

Question 3

Quelle méthode utiliser pour exclure certains objets d'un QuerySet en fonction de critères spécifiques ?

- filter()
- exclude()
- select()
- reject()

Question 4

Comment combiner des conditions avec l'opérateur OR en utilisant Q ?

- Q(a) or Q(b)
- Q(a) . Q(b)
- Q(a) | Q(b)
- Q(a) + Q(b)

Question 5

Tous droits réservés à STUDI - Reproduction interdite

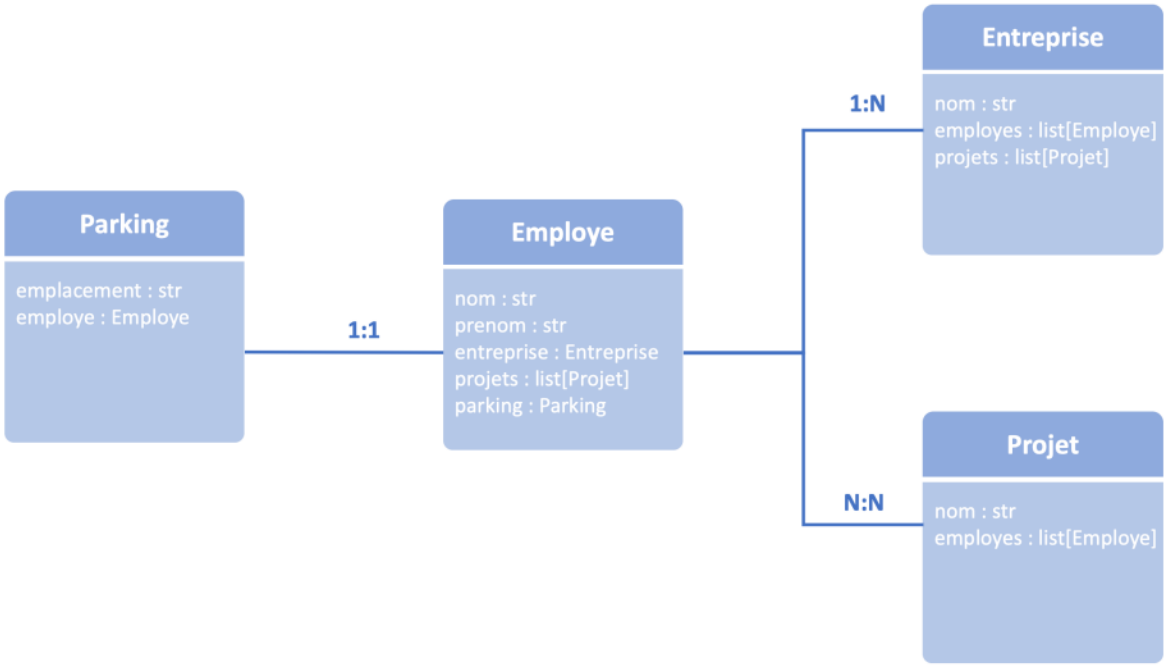
Pourquoi utiliser Q au lieu de plusieurs filter() successifs ?

- Pour éviter des requêtes SQL multiples
- Pour améliorer la lisibilité du code
- Pour permettre des conditions plus complexes comme les OR
- Toutes les réponses ci-dessus

IV. Relations multiples entre les modèles

A. Utiliser des clés étrangères

L'exploitation de relations entre différents modèles est une caractéristique clé des bases de données relationnelles. Dans Django, nous pouvons exprimer ces relations dans nos modèles en utilisant des champs spécifiques tels que ForeignKey, OneToOneField et ManyToManyField.



Dans cet exemple, un employé travaille pour une entreprise. Dans l'autre sens de la relation, une entreprise peut avoir plusieurs employés. Ce type de relation (un à plusieurs) est généralement représenté en base de données par une clé étrangère (foreign key) du côté de l'entité « *unique* » (ici en l'occurrence, Employé).

Un employé se voit attribuer une place de parking, et une place de parking n'appartient qu'à un seul employé. Cette relation « *un à un* » (one to one) est également représentée par une clé étrangère en base de données, cependant dans ce cas précis, la clé étrangère peut être attribuée d'un côté comme de l'autre de la relation. Il faudra simplement décider quel côté de la relation est le plus « *important* ».

Enfin, un employé peut participer à plusieurs projets, et un projet peut être constitué de plusieurs employés. Nous avons-là un exemple de relation « *plusieurs à plusieurs* » (many to many). Ce type de relation est généralement représenté en base de données par une table d'association (ou table pivot). Cette notion n'existe pas avec Django ORM, grâce à la couche d'abstraction apporté par celui-ci.

Imaginons que nous souhaitons maintenant créer un modèle Cours pour représenter un cours animé par un professeur. Un professeur peut animer plusieurs cours, mais chaque cours est animé par un seul professeur.

Pour modéliser cette relation « *un à plusieurs* », nous allons utiliser une ForeignKey.

Tous droits réservés à STUDI - Reproduction interdite

```
class Cours(models.Model):

    intitule = models.CharField(max_length=255)
    date_heure = models.DateTimeField(auto_now_add=True)
    professeur = models.ForeignKey(Professeur, null=True, on_delete=models.SET_NULL)
```

Le champ *intitule* est un CharField qui stockera l'intitule du cours.

date_heure est un DateTimeField qui stockera la date et l'heure du cours. L'option auto_now_add=True signifie que la date et l'heure seront automatiquement définies au moment de la création du cours si aucune autre valeur n'est transmise.

Le champ *professeur* est une ForeignKey vers le modèle Professeur. Cela signifie qu'il y a une relation « *un à plusieurs* » entre Professeur et Cours.

L'option `on_delete=models.SET_NULL` signifie que si le professeur associé est supprimé, les cours qu'il a animé seront conservés, mais plus aucun professeur n'y sera associé. Cela est possible grâce au `null=True` que nous avons défini.

La création d'un modèle avec une ForeignKey implique de déterminer quels champs sont nécessaires, de choisir le type de champ approprié pour chaque donnée à stocker, d'établir la relation ForeignKey vers le modèle pertinent, et de définir le comportement à adopter lorsque l'enregistrement lié est supprimé.

B. Relations plusieurs-à-plusieurs

Dans le *Cours* précédemment créé, nous allons ajouter une relation de type « *plusieurs à plusieurs* » avec le modèle *Etudiant*.

Un cours est constitué de plusieurs étudiants, et un étudiant peut participer à plusieurs cours.

En base de données, ce type de relation est matérialisé par une table d'association. Nous n'aurons pas à gérer cette complexité grâce à la couche d'abstraction apportée par Django ORM.

```
class Cours(models.Model):

    intitule = models.CharField(max_length=255)
    date_heure = models.DateTimeField(auto_now_add=True)
    professeur = models.ForeignKey(Professeur, null=True, on_delete=models.SET_NULL)
    etudiants = models.ManyToManyField(Etudiant)
```

C. Gérer les modèles via l'interface d'administration

Dans le fichier *admin.py* de l'application, nous avons la possibilité d'enregistrer les différents modèles créés précédemment dans l'interface administration.

Cela nous permet notamment de mettre en œuvre les actions CRUD sur ces modèles (Create, Read, Update, Delete).

```
from django.contrib import admin

from .models import *

admin.site.register(Etudiant)
admin.site.register(Professeur)
admin.site.register(Cours)
```

Tous droits réservés à STUDI - Reproduction interdite

D. Exercice : Quiz

[solution n°3 p.19]

Question 1

Que signifie l'option on_delete=models.CASCADE dans une relation ForeignKey ?

- Si l'objet lié est supprimé, rien ne se passe aux objets qui le référencent
- Si l'objet lié est supprimé, une cascade d'erreur est générée
- Si l'objet lié est supprimé, il devient NULL dans les objets qui le référencent
- Si l'objet lié est supprimé, tous les objets qui le référencent seront également supprimés

Question 2

Comment accéder aux objets liés par une ForeignKey dans une vue Django ?

- Utiliser related_name pour accéder aux objets liés depuis l'objet parent
- Utiliser reverse() pour accéder aux objets liés
- Utiliser filter() uniquement
- Utiliser ForeignKey.all()

Question 3

Est-il possible de définir plusieurs ForeignKey pointant vers le même modèle dans un seul modèle Django ?

- Oui, mais chaque ForeignKey doit avoir un nom différent
- Non, cela provoquerait des conflits
- Oui, mais seulement si les modèles sont dans la même application
- Non, Django ne permet pas cette structure

Question 4

À quoi correspond le paramètre « *null* » dans les diffférents Field ?

- Indique que le champ est toujours NULL
- Indique que le champ est NULL par défaut
- Indique si le champ peut contenir ou non la valeur NULL
- Ce paramètre n'existe pas

Question 5

Que devez-vous faire pour afficher un modèle dans l'interface d'administration de Django ?

- Importer le modèle dans le fichier views.py
- Enregistrer le modèle dans le fichier admin.py en utilisant admin.site.register()
- Ajouter le modèle au fichier urls.py
- Modifier les paramètres dans le fichier settings.py

Tous droits réservés à STUDI - Reproduction interdite

V. L'essentiel

Dans l'étude de Django, il est indispensable de comprendre les concepts fondamentaux de la modification et de la réécriture des modèles.

Les modèles sont le cœur de toute application Django : ils définissent la structure des données que votre application traite. Savoir comment manipuler et modifier ces modèles est essentiel pour gérer l'évolution de votre application avec le temps.

Que ce soit l'ajout de nouveaux champs pour collecter des données supplémentaires, la modification des options de champ existantes pour répondre à de nouveaux besoins de données, ou la suppression de champs qui ne sont plus nécessaires, chaque action a des implications significatives sur vos données existantes et le fonctionnement de votre application.

En plus de cela, comprendre le processus de migration et son importance pour synchroniser les modifications du modèle avec la base de données est essentiel.

Que ce soit l'application de nouvelles migrations pour effectuer des modifications de modèle ou l'exécution de rollbacks pour annuler les modifications, chaque étape doit être effectuée avec soin pour éviter des conséquences indésirables telles que la perte de données.

En somme, la maîtrise des modèles et des migrations dans Django est une compétence essentielle pour tout développeur Django.

VI. Auto-évaluation

A. Exercice

Vous devez développer une application de blog avec Django. Vous avez initialement défini le modèle de votre « *Post* » comme suit :

```
1 from django.db import models
2     class Post(models.Model):
3     titre = models.CharField(max_length=100)
4     contenu = models.TextField()
5     date_publiée = models.DateTimeField(auto_now_add=True)
```

Question 1

[solution n°4 p.20]

Vous décidez maintenant d'ajouter une nouvelle fonctionnalité à votre blog : la possibilité d'ajouter des tags aux posts. Pour ajouter un champ « *tags* » qui est une liste de chaînes de caractères, vous devez modifier le modèle « *Post* » .

Question 2

[solution n°5 p.21]

Une fois le modèle « *Post* » modifié, quelle commande pouvez-vous utiliser pour que la base de données prenne en compte cette modification ?

Question 3

[solution n°6 p.21]

Comment procéderiez-vous pour associer chaque « *Post* » à un « *Auteur* » ? Chaque auteur peut avoir plusieurs posts, mais chaque post ne peut avoir qu'un seul auteur. Quels changements apporteriez-vous au modèle « *Post* »pour inclure cette nouvelle fonctionnalité ?

B. Test

Exercice 1 : Quiz

[solution n°7 p.22]

Question 1

Tous droits réservés à STUDI - Reproduction interdite

Quel est le rôle principal des modèles dans une application Django ?

- Définir les routes URL de l'application
- Gérer la configuration des paramètres du projet
- Définir la structure des données et la logique métier en lien avec la base de données
- Configurer l'interface d'administration

Question 2

Quel champ Django utiliseriez-vous pour stocker du texte de longueur illimitée ?

- models.CharField()
- models.TextField()
- models.IntegerField()
- models.UnlimitedField()

Question 3

Que se passe-t-il si vous omettez d'ajouter une clé primaire dans un modèle Django ?

- Django ne permet pas de créer le modèle
- Django génère automatiquement une clé primaire appelée id
- Le modèle ne peut pas être utilisé dans les vues
- La base de données retourne une erreur lors de la migration

Question 4

Quel champ utiliseriez-vous pour créer une relation plusieurs-à-plusieurs entre deux modèles ?

- models.ForeignKey()
- models.MultipleToMultipleField()
- models.NToNField()
- models.ManyToManyField()

Question 5

Quelle méthode permet d'obtenir une représentation lisible d'un objet de modèle en Django ?

- __init__()
- __str__()
- __print__()
- __eq__()

Question 6

Tous droits réservés à STUDI - Reproduction interdite

Quelle option permet de définir un champ comme étant facultatif dans un modèle Django ?

- required=False
- null=True
- default=None
- optional=True

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 6 Solution n°1

Question 1

Qu'est-ce qui est représenté par une classe dans un modèle Django ?

- ○ Une requête à la base de données
- ○ Une colonne dans une table de la base de données
- ◉ Une table dans la base de données

Q Dans Django, chaque classe que vous définissez dans un modèle représente une table de base de données

Question 2

Dans la définition d'un modèle Django, que représente un attribut de classe ?

- ○ Un enregistrement dans la table de la base de données
- ◉ Une colonne dans la table de la base de données
- ○ Une base de données entière

Q Chaque attribut de classe que vous définissez dans un modèle Django représente une colonne spécifique dans la table de la base de données correspondante

Question 3

Quelle est la première commande que vous exécuterez après avoir créé ou modifié vos modèles Django ?

- ◉ makemigrations
- ○ migrate
- ○ syncdb

Q Après avoir créé ou modifié un modèle Django, vous utiliserez généralement la commande makemigrations. Cette commande génère les fichiers de migration nécessaires pour appliquer les modifications que vous avez apportées à la base de données.

Question 4

Comment pouvez-vous ajouter un champ de numéro de téléphone à un modèle d'Utilisateur existant ?

- ◉ En ajoutant une nouvelle instance de models.CharField au modèle Utilisateur
- ○ En modifiant le champ email pour y inclure un numéro de téléphone
- ○ En supprimant le champ email et en le remplaçant par un champ de numéro de téléphone

Q Pour ajouter un nouveau champ à un modèle existant, vous devez simplement définir un nouvel attribut de classe dans la définition du modèle. Par exemple, pour ajouter un champ de numéro de téléphone, vous pouvez ajouter une nouvelle instance de models.CharField au modèle Utilisateur.

Question 5

Qu'est-ce qu'un rollback dans le contexte des migrations de Django ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Une façon de créer une nouvelle base de données
- ◉ Un moyen d'annuler les modifications apportées à la base de données par une migration
- ○ Un moyen de sauvegarder votre base de données

Un rollback dans le contexte des migrations de Django est un moyen d'inverser les modifications apportées à la base de données par une migration. C'est une fonctionnalité utile en cas d'erreur ou de problème avec une migration, vous permettant de revenir à un état précédent de la base de données.

Question 6

Quelle est la commande pour effectuer un rollback à une migration spécifique dans Django ?

- ◉ python manage.py migrate <nom_app> <nom_migration>
- ○ python manage.py rollback <nom_app> <nom_migration>
- ○ python manage.py revert <nom_app> <nom_migration>

Pour effectuer un rollback à une migration spécifique dans Django, vous utilisez la commande python manage.py migrate <nom_app> <nom_migration>. Cette commande inverse les modifications apportées à la base de données jusqu'à la migration spécifiée.

Exercice p. 9 Solution n°2

Question 1

Quel est le rôle principal d'un QuerySet dans Django ?

- ◉ Représenter une requête SQL sur un modèle
- ○ Générer des formulaires automatiquement
- ○ Créer des vues basées sur des classes
- ○ Gérer les fichiers statiques

Un QuerySet est une collection d'objets issus de la base de données, correspondant à une requête SQL sur un modèle particulier. C'est la manière principale dont Django interagit avec la base de données.

Question 2

Quelle méthode est utilisée pour récupérer tous les objets d'un modèle sous forme de QuerySet ?

- ○ get()
- ◉ all()
- ○ filter()
- ○ select()

La méthode all() retourne un QuerySet contenant tous les objets d'un modèle, permettant ainsi de les manipuler ou de les afficher dans la vue.

Question 3

Quelle méthode utiliser pour exclure certains objets d'un QuerySet en fonction de critères spécifiques ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ filter()
- ◉ exclude()
- ○ select()
- ○ reject()

La méthode exclude() crée un QuerySet contenant tous les objets sauf ceux qui correspondent aux critères spécifiés, permettant d'exclure des résultats non désirés.

Question 4

Comment combiner des conditions avec l'opérateur OR en utilisant Q ?

- ○ Q(a) or Q(b)
- ○ Q(a) . Q(b)
- ◉ Q(a) | Q(b)
- ○ Q(a) + Q(b)

Le symbole | (pipe) est utilisé avec Q pour combiner des conditions avec l'opérateur logique OR, ce qui permet de retourner des objets qui remplissent l'une ou l'autre des conditions.

Question 5

Pourquoi utiliser Q au lieu de plusieurs filter() successifs ?

- ○ Pour éviter des requêtes SQL multiples
- ○ Pour améliorer la lisibilité du code
- ○ Pour permettre des conditions plus complexes comme les OR
- ◉ Toutes les réponses ci-dessus

Utiliser Q permet d'écrire des requêtes plus complexes avec des conditions combinées tout en gardant un code lisible et en évitant de multiplier les requêtes SQL.

Exercice p. 12 Solution n°3

Question 1

Que signifie l'option on_delete=models.CASCADE dans une relation ForeignKey ?

- ○ Si l'objet lié est supprimé, rien ne se passe aux objets qui le référencent
- ○ Si l'objet lié est supprimé, une cascade d'erreur est générée
- ○ Si l'objet lié est supprimé, il devient NULL dans les objets qui le référencent
- ◉ Si l'objet lié est supprimé, tous les objets qui le référencent seront également supprimés

L'option on_delete=models.CASCADE signifie que si l'objet auquel la clé étrangère fait référence (l'objet « *parent* ») est supprimé, alors tous les objets qui ont une ForeignKey vers cet objet (les objets « *enfants* ») seront également supprimés. C'est ce qu'on appelle une suppression en cascade.

Question 2

Comment accéder aux objets liés par une ForeignKey dans une vue Django ?

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ Utiliser related_name pour accéder aux objets liés depuis l'objet parent
- ○ Utiliser reverse() pour accéder aux objets liés
- ○ Utiliser filter() uniquement
- ○ Utiliser ForeignKey.all()

Q Le related_name dans une ForeignKey permet d'accéder aux objets liés depuis l'objet parent en utilisant une relation inverse.

Question 3

Est-il possible de définir plusieurs ForeignKey pointant vers le même modèle dans un seul modèle Django ?

- ⦿ Oui, mais chaque ForeignKey doit avoir un nom différent
- ○ Non, cela provoquerait des conflits
- ○ Oui, mais seulement si les modèles sont dans la même application
- ○ Non, Django ne permet pas cette structure

Q Il est tout à fait possible de définir plusieurs ForeignKey pointant vers le même modèle dans un seul modèle Django, à condition que chaque champ ForeignKey ait un nom différent pour éviter les conflits.

Question 4

À quoi correspond le paramètre « *null* » dans les diffférents Field ?

- ○ Indique que le champ est toujours NULL
- ○ Indique que le champ est NULL par défaut
- ⦿ Indique si le champ peut contenir ou non la valeur NULL
- ○ Ce paramètre n'existe pas

Q Le paramètre null=True permet à un champ de contenir une valeur NULL dans la base de données, ce qui est utile pour les champs non obligatoires.

Question 5

Que devez-vous faire pour afficher un modèle dans l'interface d'administration de Django ?

- ○ Importer le modèle dans le fichier views.py
- ⦿ Enregistrer le modèle dans le fichier admin.py en utilisant admin.site.register()
- ○ Ajouter le modèle au fichier urls.py
- ○ Modifier les paramètres dans le fichier settings.py

Q Pour afficher un modèle dans l'interface d'administration, il est nécessaire de l'enregistrer dans le fichier admin.py en utilisant la fonction admin.site.register().

p. 13 **Solution n°4**

Tous droits réservés à STUDI - Reproduction interdite

Pour ajouter la fonctionnalité de tags à votre modèle « *Post* » dans Django, vous pouvez utiliser le champ ManyToManyField. Ce champ est idéal pour représenter une relation « *plusieurs à plusieurs* ».

```
1 from django.db import models
2
3     class Tag(models.Model):
4         nom = models.CharField(max_length=50)
5
6     class Post(models.Model):
7         titre = models.CharField(max_length=100)
8         contenu = models.TextField()
9         date_publiée = models.DateTimeField(auto_now_add=True)
10         tags = models.ManyToManyField(Tag)
```

Nous venons de créer un nouveau modèle « *Tag* » qui représente les tags. Ensuite, nous avons inclus un champ « *tags* » à notre modèle « *Post* » afin de créer une relation de plusieurs à plusieurs entre « *Post* »et « *Tag* ».

Ici, nous avons créé un nouveau modèle « *Tag* » pour représenter les tags. Ensuite, nous avons ajouté un champ « *tags* » à notre modèle « *Post* » qui crée une relation de plusieurs à plusieurs entre « *Post* » et « *Tag* ».

p. 13 Solution n°5

Une fois que vous avez apporté des modifications au modèle « *Post* », vous pouvez prendre en compte ces changements dans la base de données en créant une migration pour ces modifications. Cela peut être fait en utilisant la commande makemigrations suivie du nom de votre application, par exemple :

```
1 python manage.py makemigrations nom_de_votre_application
```

Ensuite, appliquez cette migration à votre base de données en utilisant la commande migrate :

```
1 python manage.py migrate
```

Ces commandes mettent à jour votre base de données pour refléter les modifications apportées à votre modèle « *Post* ».

p. 13 Solution n°6

Vous pouvez vous servir du champ ForeignKey de Django qui permet de créer une relation de « *plusieurs à un* » entre les modèles.

```
1 from django.db import models
2
3 class Auteur(models.Model):
4     nom = models.CharField(max_length=100)
5     email = models.EmailField()
6
7 class Tag(models.Model):
8     nom = models.CharField(max_length=50)
9
10 class Post(models.Model):
11     titre = models.CharField(max_length=100)
12     contenu = models.TextField()
13     date_publie = models.DateTimeField(auto_now_add=True)
14     tags = models.ManyToManyField(Tag)
15     auteur = models.ForeignKey(Auteur, on_delete=models.CASCADE)
```

Tous droits réservés à STUDI - Reproduction interdite

Ici, nous avons ajouté un nouveau modèle « *Auteur* » pour représenter les auteurs. Ensuite, nous avons ajouté un champ « *auteur* » à notre modèle « *Post* » qui crée une relation « *plusieurs à un* » entre « *Post* » et « *Auteur* ». L'option on_delete=models.CASCADE signifie que lorsque l'auteur associé est supprimé, tous les posts liés à cet auteur seront également supprimés.

N'oubliez pas, après avoir ajouté le champ ForeignKey, vous devez à nouveau faire une migration pour ces modifications.

Exercice p. 13 Solution n°7

Question 1

Quel est le rôle principal des modèles dans une application Django ?

- ○ Définir les routes URL de l'application
- ○ Gérer la configuration des paramètres du projet
- ◉ Définir la structure des données et la logique métier en lien avec la base de données
- ○ Configurer l'interface d'administration

Q Les modèles en Django définissent la structure des données et la logique métier associée, en mappant les classes Python à des tables de base de données.

Question 2

Quel champ Django utiliseriez-vous pour stocker du texte de longueur illimitée ?

- ○ models.CharField()
- ◉ models.TextField()
- ○ models.IntegerField()
- ○ models.UnlimitedField()

Q models.TextField() est utilisé pour stocker du texte de longueur illimitée, contrairement à CharField qui a une longueur maximale définie.

Question 3

Que se passe-t-il si vous omettez d'ajouter une clé primaire dans un modèle Django ?

- ○ Django ne permet pas de créer le modèle
- ◉ Django génère automatiquement une clé primaire appelée id
- ○ Le modèle ne peut pas être utilisé dans les vues
- ○ La base de données retourne une erreur lors de la migration

Q Si une clé primaire n'est pas explicitement définie, Django crée automatiquement un champ id en tant que clé primaire.

Question 4

Quel champ utiliseriez-vous pour créer une relation plusieurs-à-plusieurs entre deux modèles ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ models.ForeignKey()
- ○ models.MultipleToMultipleField()
- ○ models.NToNField()
- ◉ models.ManyToManyField()

models.ManyToManyField() est utilisé pour établir une relation plusieurs-à-plusieurs entre deux modèles.

Question 5

Quelle méthode permet d'obtenir une représentation lisible d'un objet de modèle en Django ?

- ○ __init__()
- ◉ __str__()
- ○ __print__()
- ○ __eq__()

La méthode __str__() est utilisée pour renvoyer une représentation lisible de l'objet lorsqu'il est converti en chaîne de caractères.

Question 6

Quelle option permet de définir un champ comme étant facultatif dans un modèle Django ?

- ○ required=False
- ◉ null=True
- ○ default=None
- ○ optional=True

Utiliser null=True permet de stocker des valeurs NULL dans la base de données, tandis que blank=True permet de rendre le champ facultatif dans les formulaires.

Tous droits réservés à STUDI - Reproduction interdite